

EnTiSe: A Python framework for scalable synthetic energy time-series generation

Markus Doepfert*, Thomas Hamacher

Technical University of Munich, TUM School of Engineering and Design, Chair for Renewable and Sustainable Energy Systems, Lichtenbergstr. 4a, 85748 Garching, Germany

ARTICLE INFO

Keywords:

Python
Energy system modelling
Synthetic time series
Bulk generation
Building simulation
Scalable workflow

ABSTRACT

Energy system models often rely on synthetic time series due to the limited availability of high-resolution input data. Existing Python tools provide domain-specific generators but differ in interfaces, assumptions, and scalability. EnTiSe is a lightweight Python framework installable through PyPI for generating synthetic energy time series via a unified, extensible API. It supports bulk data generation, enabling the scalable creation of interdependent time series across domains such as heating, cooling, electricity, and renewable energy generation. Its abstract base class architecture allows seamless integration of new methods while preserving a consistent workflow. Benchmark results demonstrate efficient large-scale generation suitable for building stock simulations and energy system studies.

Metadata

Code metadata (mandatory).

Nr.	Code metadata description	Please fill in this column
C1	Current code version	v1.2.0
C2	Permanent link to code/repository used for this code version	https://github.com/tum-ens/EnTiSe/tree/v1.2.0
C3	Permanent link to Reproducible Capsule	—
C4	Legal Code License	MIT
C5	Code versioning system used	git
C6	Software code languages, tools, and services used	Python
C7	Compilation requirements, operating environments & dependencies	Python ≥ 3.10
C8	If available Link to developer documentation/manual	https://entise.readthedocs.io
C9	Support email for questions	markus.doepfert@tum.de

Software metadata (optional).

Nr.	(Executable) software metadata description	Please fill in this column
S1	Current software version	v1.2.0
S2	Permanent link to executables of this version	https://github.com/tum-ens/EnTiSe/tree/v1.2.0
S3	Permanent link to Reproducible Capsule	—
S4	Legal Software License	MIT
S5	Computing platforms/Operating Systems	iOS, Linux, Microsoft Windows
S6	Installation requirements & dependencies	Python ≥ 3.10
S7	If available, link to user manual - if formally published include a reference to the publication in the reference list	https://entise.readthedocs.io
S8	Support email for questions	markus.doepfert@tum.de

* Corresponding author.

Email address: markus.doepfert@tum.de (M. Doepfert).

Table 1

Existing Python packages and tools and the number of methods they support. Abbreviations: HVAC = heating, ventilation, and air conditioning; DHW = domestic hot water; HP = heat pump; PV = photovoltaics.

Name	Package	HVAC	Electricity	Mobility	DHW	Occupancy	PV	Wind	HP	Source
ALPG	✗	–	1	–	–	–	–	–	–	[6]
atlite	✓	1	–	–	–	–	1	1	–	[7]
Demandlib	✓	2	2	–	1	–	–	–	–	[8]
Demod	✓	1	–	–	–	–	–	–	–	[9]
DistrictHeatingSim	✓	1	–	–	–	–	–	–	–	[10]
emobpy	✓	–	–	1	–	–	–	–	–	[11]
ETHOS.PeNALPS	✗	–	1	–	–	–	–	–	–	[12]
ETHOS.TISED	✗	–	–	–	–	–	1	–	–	[1]
EURECA	✗	2	–	–	–	–	–	–	–	[13]
EV-SDG	✗	–	–	1	–	–	–	–	–	[14]
Heatpro	✓	1	–	–	–	–	–	–	–	[15]
HEDGE	✗	–	1	1	–	–	1	–	–	[16]
hplib	✓	–	–	–	–	–	–	–	1	[17]
OpenDHW	✗	–	–	–	1	–	–	–	–	[18]
pvlb	✓	–	–	–	–	–	1	–	–	[19]
PyLPG	✓	1	1	–	–	1	–	–	–	[20]
RAMP	✓	–	1	1	1	–	–	–	–	[21]
SPAGHETTI	✗	–	–	1	–	–	–	–	–	[22]
TEASER	✓	1	–	–	–	–	–	–	–	[23]
tsib	✓	1	1	–	1	1	–	–	–	[24]
Vencopy	✓	–	–	1	–	–	–	–	–	[25]
windpowerlib	✓	–	–	–	–	–	–	1	–	[26]
EnTiSe	✓	5	2	–	1	2	1	1	1	This paper

1. Motivation and significance

Energy system models depend on high-quality input data, yet detailed empirical time series are often unavailable at the required spatial or temporal resolution [1]. As a result, synthetic time series generation has become a standard component of many modelling workflows. Numerous Python packages provide domain-specific generators for applications such as building thermal demand, photovoltaic output, or electricity consumption, as summarized in Table 1.

While these tools offer valuable functionality within their respective domains, the ecosystem remains structurally fragmented. Existing approaches typically exhibit one or more of the following limitations:

1. Tools are developed independently and expose heterogeneous APIs, requiring researchers to learn multiple interface conventions and manually align inputs and outputs.
2. Several packages are designed as standalone frameworks rather than installable Python packages, complicating integration into reproducible workflows.
3. Some implementations provide highly detailed and specialized functionality that exceeds the needs of many energy system modellers, increasing model complexity and computational cost.
4. Time series are often generated as isolated artefacts, whereas many applications require coherent sets of time series across multiple domains—for example, occupancy patterns, appliance electricity consumption, and thermal loads that share a common building context—to be produced and varied consistently across large numbers of objects. Existing tools do not support this multi-domain coherence natively, requiring users to implement custom coupling logic.
5. Large-scale generation across hundreds or thousands of entities is not a primary design objective of most tools, making bulk processing cumbersome or inefficient.

Consequently, researchers who require heterogeneous and interdependent time series, such as in agent-based energy market models or large-scale building stock simulations [2–5], must manually combine multiple tools, harmonize interfaces, and implement custom

orchestration logic. The effort required for data generation can therefore become a substantial share of the overall modelling workload.

EnTiSe addresses this architectural gap by providing a lightweight Python framework for the coherent, scalable generation of synthetic energy time series via a unified, extensible interface. Rather than introducing a new domain-specific model, EnTiSe establishes an object-centric bulk generation workflow that integrates heterogeneous methods while preserving a consistent execution model.

2. Software description

2.1. Software architecture

EnTiSe was designed for usability and extensibility. To achieve this, the architecture shown in Fig. 1 was kept simple, ensuring that new methods require only a single additional module without modifying any part of the core architecture.

Two usage modes are supported, *method-centric* and *object-centric*. In the *method-centric* approach, users access the time series methods (e.g., the 1R1C (one thermal resistance, one thermal capacitance) method) directly. In the *object-centric* approach, they create a tabular object that contains the information needed to generate several time series at once. The former is more suitable for single time series with greater control needs, as illustrated in Listing 1. The latter is intended for bulk generation, e.g., when calculating heating, ventilation, and air conditioning (HVAC) demands for entire cities or countries.

If the user aims to generate in bulk (*object-centric*), they first create the information about the time series that are to be created. This can occur outside of EnTiSe and then be loaded using DataFrames or directly in EnTiSe using dictionaries or DataFrames. Then, an instance of the Generator class (see Fig. 1) needs to be created, and the object information is added after instantiation using the `add_objects()` method. Finally, the time series generation can be started using the `generate()` command. The Generator handles the bulk generation using `joblib` [27] as well as result retrieval. The entire process is illustrated in Listing 2.

The Runner class is called by the Generator class for every time series object. Objects are defined here as entities for which several time series need to be calculated. This can, for example, be a building that requires

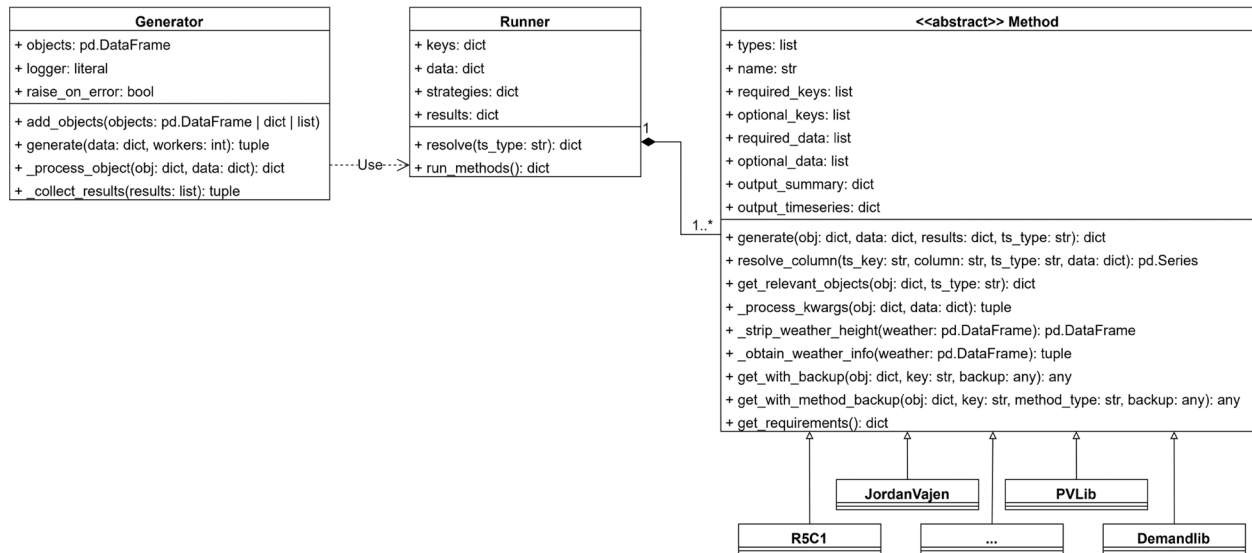


Fig. 1. Core architecture of EnTiSe. All user-facing methods extend the abstract Method base class, which guarantees a consistent interface. The MethodMeta registry (not shown) auto-registers each subclass by name, enabling dynamic dispatch. The Runner orchestrates method calls for individual objects, and the Generator handles bulk parallel execution via joblib.

```

from entise.methods.hvac import R5C1
import pandas as pd
object = {"id": "bldg_1",
          "resistance[K W-1]": 5e-3,
          "capacitance[J K-1]": 1e8}
weather_df = pd.read_csv('weather.csv') # time-indexed
      DataFrame with outdoor temperature
ts, summary = R5C1().generate(
    obj=object,
    weather=weather_df
)
  
```

Listing 1. Method-centric usage of EnTiSe.

time series for electricity, HVAC, and heat pump, or a site where the potential for solar and wind needs to be calculated. Runner calls the required methods sequentially by first checking in the *registry* that the method exists. If methods depend on each other, it uses dependency resolution based on declared required keys/data to call the methods in an appropriate order.

All methods are derived from an abstract base class (ABC) called Method. This ensures conformity and that the Generator class can be used for bulk generation. The Method class requires every method to define the following:

- **types:** a list containing the types of time series the method can create
- **name:** the name of the method as it can be called from the tabular object information
- **required_keys:** the keys the objects need to contain for the method to work

- **optional_keys:** any optional keys that can be provided but are not necessary, as they will be replaced by default values
- **required_data:** the necessary data that does not fit into a tabular format, e.g., dictionaries or dataframes
- **optional_data:** any optional data that is otherwise replaced with default values
- **output_summary:** all output summary values provided by the method
- **output_timeseries:** all columns of the time series dataframe that are returned by the method

Finally, every child class of Method must include a `generate` method that takes the input, calculates the time series, and returns the formatted summary and time series. Further restrictions are not included to allow as much flexibility as possible when creating new methods. This design prioritizes flexibility, while leaving responsibility for implementation quality to the developer.

```

from entise import Generator
import pandas as pd

objects = pd.read_csv('object.csv') # tabular data
    containing all object relevant information
weather_df = pd.read_csv('weather.csv') # time-indexed
    DataFrame with outdoor temperature
data = {'weather': weather_df} # keys must correspond
    with information in objects
gen = Generator()
gen.add_objects(objects)

summary, timeseries = gen.generate(data)

```

Listing 2. Object-centric usage of EnTiSe.

2.2. Software functionalities

EnTiSe provides a framework for generating synthetic energy time series across multiple demand and supply domains through a unified interface. All methods expose a consistent I/O contract by returning a time-indexed dataframe alongside a standardized set of summary values, enabling direct comparison and combination of outputs across methods. A key capability is the creation of coherent multi-domain datasets by allowing generated series to be reused as inputs within the same scenario, supporting coupled workflows such as occupancy-driven internal gains and appliance electricity influencing heating and cooling demand. Beyond the built-in methods, EnTiSe supports integration of user-defined generators that follow the same interface, allowing users to benchmark new approaches against existing implementations and to extend the framework to additional time-series types.

EnTiSe includes 13 built-in methods for generating synthetic time series across several energy-related domains. These methods cover demand-side processes (electricity, HVAC, domestic hot water, mobility, and occupancy) as well as selected supply and conversion components (photovoltaic generation, wind generation, and heat pump performance). All methods generate time series at the resolution and the duration of the given input data, e.g., the weather time series. Each method implements a distinct modelling approach for one or more time-series types and is selected by its name (*object-centric*) or by directly calling its *generate* method (*method-centric*).

While methods targeting the same domain may differ in required inputs, level of detail, or modelling assumptions, they all expose the same external interface. This allows users to substitute or compare methods without altering downstream processing and to evaluate modelling choices within otherwise identical scenarios. Table 2 summarizes the implemented methods, the time-series types they support, whether they were implemented natively or by wrapping another Python package, and references to the underlying models or data sources. EnTiSe distinguishes between *native* methods, where the algorithm is implemented directly within the framework, and *wrapper* methods, which delegate to an established third-party package. The thermal HVAC models (1R1C, 5R1C, 7R2C) [28,29], the domestic hot water profile generator [30], the heatpump [31] and the occupancy models [32] are native implementations of established methods. The photovoltaic, wind, electricity demand, and heating methods wrap *pvlb* [19], *windpowerlib* [26], *demandlib* [8], *DistrictHeatingSim* [10] and *pyLPG* [20], respectively. The *pvlb* wrapper supports fixed-tilt configurations. The *windpowerlib* wrapper applies a logarithmic wind profile to extrapolate wind speed to hub height and uses power-curve-based modelling. Large-scale wake

Table 2

Overview of methods implemented in EnTiSe, their supported time-series types, implementation approach, and references.

Method	Type	Implementation	Reference
1R1C	HVAC	Native implementation	[34]
5R1C	HVAC	Native implementation	[28]
7R2C	HVAC	Native implementation	[29]
DemandLib	Electricity	Wrapped Python package	[8]
DemandLib	HVAC	Wrapped Python package	[8]
JordanVajen	DHW	Native implementation	[30]
DistrictHeatingSim	HVAC	Wrapped Python package	[10]
File	All	Native implementation	–
GeoMA	Occupancy	Native implementation	[32]
PHT	Occupancy	Native implementation	[32]
pvlb	PV	Wrapped Python package	[19]
pyLPG	Electricity	Wrapped Python package	[20]
Ruhnau	Heat pump	Native implementation	[31]
windpowerlib	Wind	Wrapped Python package	[26]

loss modelling is not included, as it depends on site-specific layouts beyond the scope of single-turbine time-series generation. Both wrappers have been validated against the respective upstream packages' own test suites. EnTiSe currently works with any weather dataset compatible with the underlying libraries, e.g., [33]. Bias adjustment of input weather data is not performed within EnTiSe and should be applied upstream if required.

Listing 3 illustrates the API reduction EnTiSe provides: a photovoltaic time series that requires approximately 10 lines of *pvlb* setup code can be generated through EnTiSe with a single *generate()* call, while remaining fully compatible with the same underlying *ModelChain* computation. The reduction is even more beneficial if several time series are created, e.g., PV, electricity demand and heating demand, and various calls need to be made as EnTiSe methods are always called using the same API style (*generate()*) or using the tabular format for bulk generation.

EnTiSe returns generated time series and summary information in a structured and deterministic format. Time series outputs are provided as a nested dictionary, where the top-level keys correspond to object identifiers and each object maps to a dictionary of time-series types (e.g., HVAC, mobility), with each entry containing a pandas dataframe indexed by timestamps. All time series dataframes use a *DatetimeIndex*. While different temporal resolutions across methods are technically supported, a consistent resolution within a scenario is recommended to ensure coherent downstream use. In addition to the time series outputs,

```
# Direct pvlib usage
import pvlib

location = pvlib.location.Location(latitude=48.1,
                                   longitude=11.5, altitude=520)
system = pvlib.pvsystem.PVSystem(surface_tilt=30,
                                  surface_azimuth=180,
                                  module_parameters=pvlib.pvsystem.retrieve_sam('
                                      SandiaMod'))['...'],
                                  inverter_parameters=pvlib.pvsystem.retrieve_sam('
                                      cecinverter'))['...'])
mc = pvlib.modelchain.ModelChain(system, location)
mc.run_model(weather_df)
result = mc.results.ac

# EnTiSe wrapper
from entise.methods.pv import PVLib
_, ts = PVLib().generate(latitude=48.1, longitude=11.5,
                          tilt=30, azimuth=180,
                          weather=weather_df)
```

Listing 3. Comparison of direct pvlib usage (top) versus the EnTiSe wrapper (bottom) for generating a PV time series.

EnTiSe returns a separate summary dataframe with one row per object, where summary values are method-specific and reflect aggregated characteristics such as annual demand, peak values, and performance indicators. Within a given object, each time series type is generated by at most one method, and methods may return multiple columns per dataframe (e.g., heating power, cooling power, and indoor temperature).

EnTiSe is designed to be extensible, allowing users to integrate custom time series generators alongside the built-in methods while preserving a consistent interface and output structure. Custom methods follow the same conceptual workflow as native implementations and can therefore be used interchangeably within existing scenarios (see [Section 2.1](#)).

Integrating a new method into EnTiSe consists of the following steps:

1. Define the method interface by specifying the supported time-series types, required and optional inputs, and corresponding default values.
2. Implement the method as a subclass of the ABC Method interface, ensuring compatibility with the existing generation workflow.
3. Implement the generation logic that computes the time series from the provided inputs and returns outputs conforming to the standardized output structure described above.
4. Register the method such that it can be discovered and invoked by name within user-defined scenarios.
5. Validate the implementation by adding tests, documentation, and example configurations to ensure reproducibility and correct integration.

2.3. Performance & scalability

To evaluate computational performance under realistic usage conditions, all implemented methods were benchmarked using the object-centric workflow. For each method, 100 objects were processed within a

single bulk call to the `generate()` function. Reported runtimes include dependency resolution, method execution, parallelization overhead, and result aggregation. External data loading and manual preprocessing steps are excluded. Benchmarks were conducted on a laptop equipped with an AMD Ryzen 7 PRO 6850U (8 physical cores, 16 threads, 2.7–4.7 GHz), 32 GB RAM, running Windows 11 and Python 3.13.

Two execution modes were considered:

1. Single-core execution
2. Multi-core execution with 4 workers and a batch size of 25

Parallelization is implemented strictly at object level. Individual objects are processed concurrently, while each method's internal computational logic remains unchanged. Batching distributes objects evenly across workers to reduce scheduling overhead. [Fig. 2](#) summarizes the resulting runtimes. Exact numerical values are provided in the accompanying table.

Runtime differences across methods primarily reflect the intrinsic computational complexity of the underlying models. The lightweight HVAC-1R1C (one thermal resistance, one capacitance) implementation processes 100 objects in 4.58 s (21.82 iterations per second), whereas more detailed resistance-capacitance models such as HVAC-7R2C require 15.82 s. Wrapped external libraries such as PV-pvlib and wind-wplib complete in 26.77 s and 18.32 s, respectively. More computationally intensive models, including heat-districtheat, require 113.83 s for the same workload. The electricity-pyLPG method requires more than 13 h for 100 objects, reflecting the internal design of the wrapped package rather than overhead introduced by EnTiSe.

For computationally demanding methods, multi-core execution substantially reduces runtime. Observed speed-up factors between single-core and four-worker execution range from approximately 1.4× to 3.1×. The largest relative improvement is observed for heat-districtheat, which decreases from 113.83 s to 36.57 s (3.11×). PV-pvlib and

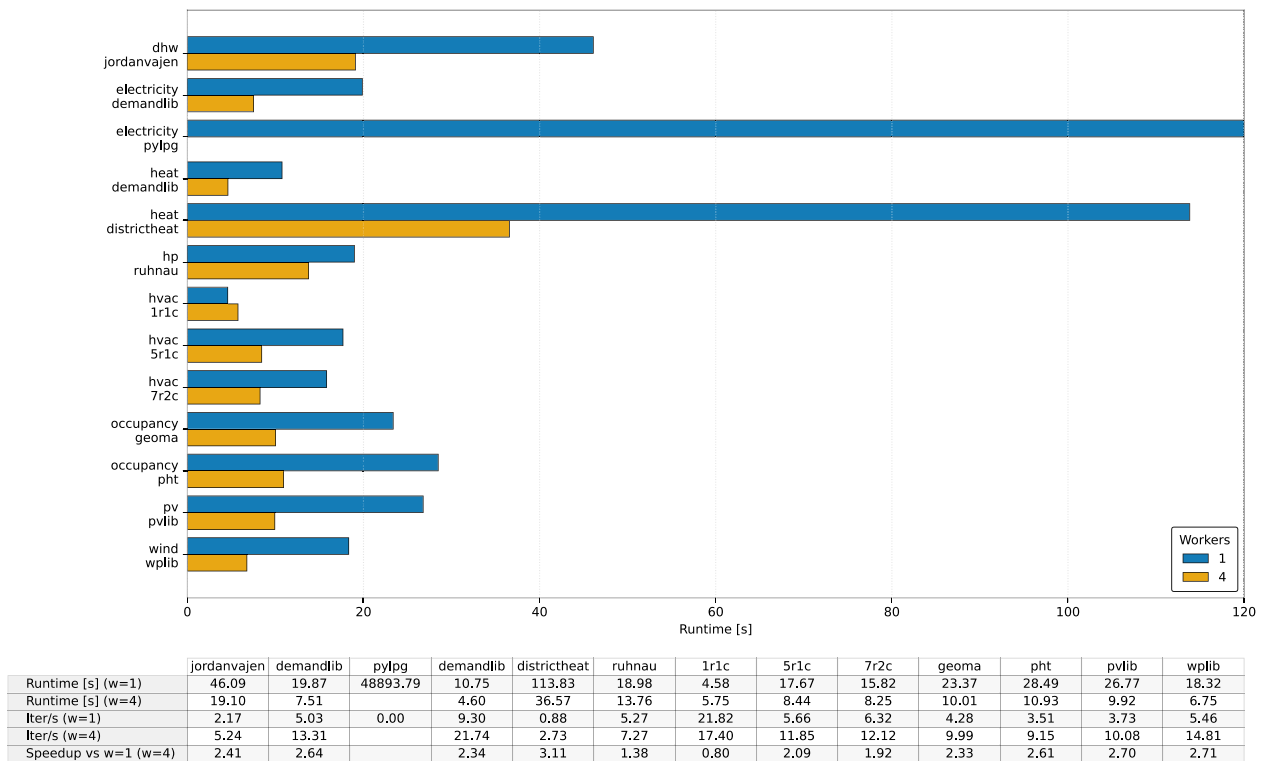


Fig. 2. Runtime comparison of implemented generation methods for 100 objects within the object-centric workflow. Each method was executed in single-core mode and in parallel using four worker processes. The y-axis is truncated at 120 seconds for readability. Exact runtime values are reported in the accompanying table. The electricity-pyLPG method is shown in single-core mode only due to internal database access constraints which prevent parallel execution.

wind-wplib achieve speed-ups of 2.70 $\times$ and 2.71 $\times$, respectively. In contrast, hp-ruhnaue shows a more moderate improvement (18.98 s to 13.76 s, 1.38 $\times$).

For very lightweight methods, multiprocessing overhead can outweigh computational gains. The HVAC-1R1C method increases slightly from 4.58 s to 5.75 s under four workers (speed-up factor 0.80), indicating that object-level parallelization becomes advantageous only once per-object computation time exceeds process management overhead.

To illustrate this effect, HVAC-1R1C was additionally benchmarked for 2000 objects using six workers. Runtime decreased from 97.30 s (single-core) to 32.77 s (parallel), corresponding to a 2.97 $\times$ speed-up. This confirms that object-level parallelization scales effectively for sufficiently large workloads.

The benchmark results indicate that:

- Runtime differences are dominated by the intrinsic complexity of the underlying models rather than architectural overhead.
- The object-centric execution layer introduces minimal computational overhead.
- Parallelization at object level provides substantial acceleration for moderate to large workloads.
- Extremely lightweight models may not benefit from multiprocessing at small batch sizes due to process management overhead.

Overall, EnTiSe enables efficient bulk generation across heterogeneous time series methods while preserving a unified execution model. Memory consumption scales approximately linearly with the number of objects, as each object's result is a fixed-size one-year hourly dataframe. Scaling beyond a single machine is not natively supported.

3. Illustrative example

To illustrate the capabilities of EnTiSe, we consider a single-family house (SFH) for which the space heating demand is generated using

Table 3

Scenario configuration for the heat pump comparison example.

Scenario	Heating demand	Heat pump source	Sink temperature
Air (radiator)	12.78 MWh	Air	50 °C
Soil (radiator)	12.78 MWh	Soil	50 °C
Water (radiator)	12.78 MWh	Water	50 °C
Air (low-temp)	12.78 MWh	Air	40 °C

the implemented 1R1C HVAC method. Based on this thermal demand, the corresponding electricity consumption is calculated using alternative heat pump configurations. The scenarios differ only in the heat pump characteristics, while building parameters, weather data, and heating demand remain identical. All time series are generated at hourly resolution over a full calendar year (8760 time steps).

Specifically, we vary the heat source (air, soil, and water) as well as the required sink temperature. The latter represents the supply temperature of the heating system and may reflect, for example, the use of conventional radiators (50 °C) or a low-temperature system (40 °C). By holding all other inputs constant, differences in the resulting electrical load profiles can be attributed solely to the selected heat pump configuration. The corresponding scenario definitions are summarized in Table 3.

Fig. 3 presents the resulting electricity demand under the four heat pump configurations. Fig. 3(a) shows the load duration curves, highlighting differences in peak load levels and the overall distribution of electricity demand. The air-source heat pump operating at a 50 °C sink temperature exhibits both the highest peak loads and the largest annual electricity consumption. In contrast, the water-source system yields a flatter curve and lower peak values, reflecting higher and more stable source temperatures compared to air-source systems. Reducing the sink temperature to 40 °C decreases electricity demand across the entire

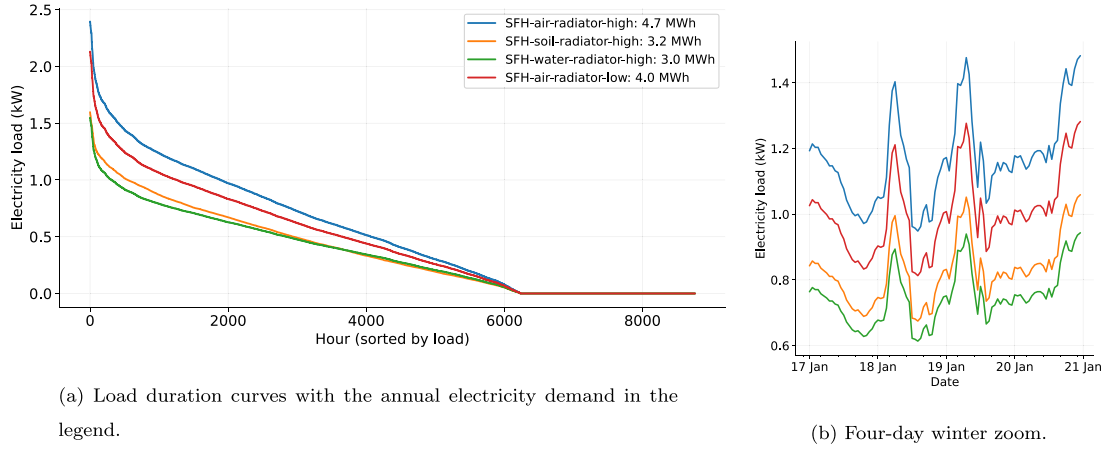


Fig. 3. Electricity demand resulting from a fixed heating demand time series combined with alternative heat pump configurations. (a) shows load duration curves; (b) shows a winter time window to illustrate temporal dynamics.

load range due to improved heat pump efficiency, although peak loads remain higher than for ground- or water-source systems.

Fig. 3(b) focuses on a representative winter period to visualize the temporal dynamics. While all scenarios follow the same temporal pattern imposed by the heating demand, the magnitude and volatility of electricity consumption differ noticeably. Air-source systems show stronger peaks during cold spells, whereas soil- and water-source configurations exhibit smoother profiles.

Without a unified framework, this comparison would require manually coupling a thermal demand model with separate heat pump performance tools, harmonizing temporal resolution, and ensuring consistent parameter handling across implementations. EnTiSe enables this type of controlled scenario analysis within a single object-centric workflow while preserving reproducibility and method interchangeability.

Listing 4 further illustrates the simplification through a unified API-call. It shows how photovoltaic and wind time series can be generated alongside the demand-side example for one object above, using the same unified interface. Note that further parameters can be passed to the methods. This example shows only the minimum information required and relies heavily on defaults. Multiple objects can either be added at once using a tabular format or sequentially by calling `gen.add_objects(object)` multiple times.

4. Impact

EnTiSe addresses a structural bottleneck in energy system modelling: the scalable and coherent generation of heterogeneous input time series. While numerous domain-specific generators exist, the absence of a unifying and extensible bulk-generation architecture has limited the practical feasibility of large-scale, interdependent data generation. EnTiSe shifts this limitation by enabling object-centric generation of consistent time series across domains within a single workflow.

A primary impact of EnTiSe lies in enabling high-resolution building stock simulations at national or regional scales. Within the German research project NEED (New Data for the Energy Transition) [35], EnTiSe is used to generate hourly heating and cooling demand time series at building level for entire federal states, corresponding to millions of objects. On standard workstation hardware, vectorized native implementations achieve up to 45 iterations per second (single-core) for resistance-capacitance models, allowing national-scale simulations to be completed within days rather than weeks. Such studies would be computationally prohibitive or operationally cumbersome if time series had to be generated using heterogeneous, standalone tools that require manual orchestration.

Beyond scale alone, EnTiSe improves the methodological flexibility of existing research workflows. In energy system studies, demand and

generation profiles are often treated as static inputs. EnTiSe enables endogenous regeneration of heating, electricity, and renewable generation time series under varying building parameters, technology assumptions, and occupancy patterns while preserving a consistent execution model. This facilitates systematic sensitivity analyses and scenario exploration without restructuring the surrounding modelling code.

The unified API further reduces the need for custom integration logic between independent Python packages. Existing tools such as `pvl`, `windpowerlib`, or `demandlib` remain integral to the ecosystem; however, their heterogeneous interfaces and conventions typically require substantial harmonization efforts. By abstracting these differences through a common interface, EnTiSe reduces workflow complexity and the technical overhead associated with multi-domain modelling.

The abstract base class architecture enables researchers to integrate custom time series generation methods without modifying the surrounding execution logic. New implementations automatically benefit from bulk processing, parallel execution, and standardized output handling. This lowers the barrier for methodological experimentation while maintaining reproducibility and comparability across studies.

EnTiSe is currently applied in doctoral research, master theses, and national-scale research projects. Its use within the NEED project demonstrates applicability at building-stock scale, where millions of hourly demand profiles are generated reproducibly within a unified workflow. In combination with `infDB` [36], EnTiSe forms part of a structured energy data pipeline linking synthetic time series generation with standardized data storage and model integration. This integration enables consistent, reproducible input generation across studies while reducing implementation effort in multi-domain energy system modelling.

5. Conclusions

This paper introduced EnTiSe, a lightweight and extensible Python framework for the scalable generation of synthetic energy time series. Rather than proposing a single domain-specific model, EnTiSe establishes an object-centric bulk-generation architecture that unifies heterogeneous methods through a consistent interface. This design enables the coherent generation of interdependent (i.e., time series whose outputs feed into subsequent computations – for example, an occupancy profile being derived from the electricity profile) time series across domains such as heating, cooling, electricity demand, and renewable generation within a single execution workflow.

Benchmark results demonstrate that the framework supports efficient large-scale generation while maintaining minimal architectural overhead. Lightweight vectorized implementations achieve high throughput on standard workstation hardware, and object-level parallelization enables further acceleration for computationally intensive

```

from entise import Generator
import pandas as pd

object = {'id': 'SFH',
          'hp': 'ruhnau',
          'hvac': '1R1C',
          'pv': 'pvlib',
          'wind': 'wplib',
          'weather': 'weather_ts', # value must match
                                   key in "data"
          'resistance[K W-1]': 5e-3,
          'capacitance[J K-1]': 1e8,
          'latitude[degree]': 49,
          'longitude[degree]': 11,
}

data = {'weather_ts': pd.read_csv('weather.csv')}

gen = Generator()
gen.add_objects(object)
summary, timeseries = gen.generate(data)

```

Listing 4. Supply-side time series generation with EnTiSe (PV and wind).

workloads. Performance differences primarily reflect the intrinsic complexity of the underlying models rather than constraints imposed by the framework.

By abstracting heterogeneous APIs and enforcing a standardized execution model, EnTiSe reduces workflow complexity and lowers the technical barrier for large-scale synthetic data generation. The abstract base class architecture allows researchers to integrate custom methods while automatically benefiting from bulk processing, parallel execution, and standardized output handling.

Future development will extend the range of supported generation methods, with planned additions including run-of-river and reservoir hydro power models, bias adjustment routines for weather inputs, and spatial interpolation of gridded climate data to arbitrary site locations. Interoperability with established tools in the Python energy modelling ecosystem will be further strengthened, covering mobility, heat pump, and renewable generation libraries.

CRediT authorship contribution statement

Markus Doeppfert: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Thomas Hamacher:** Supervision, Resources, Project administration, Funding acquisition.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

Markus Doeppfert reports that financial support was provided by Federal Ministry for Economic Affairs and Energy. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

The authors thank the German Federal Ministry for Economic Affairs and Energy, which funded the presented work through the research project NEED. We further thank Gaurav Mahesh Lad and Maximiliano Fernando Escobar for their contributions to the package.

References

- [1] Omoyele O, Hoffmann M, Weinand JM, Larrañeta M, Linßen J, Stolten D. A high-resolution downscaling approach for solar irradiance using statistical parameter matching. *Renew Energy* 2026;256:124551. <https://doi.org/10.1016/j.renene.2025.124551>. <https://www.sciencedirect.com/science/article/pii/S0960148125022153>.
- [2] Doeppfert M, Chu J, Hamacher T. HAMLET: a modular agent-based Python framework for energy markets and systems. *SoftwareX* 2025;32:102346. <https://doi.org/10.1016/j.softx.2025.102346>. <https://www.sciencedirect.com/science/article/pii/S2352711025003127>.
- [3] Essayeh C, Morstyn T. OPLEM: open platform for local energy markets. *Appl Energy* 2024;373:123848. <https://doi.org/10.1016/j.apenergy.2024.123848>. <https://linkinghub.elsevier.com/retrieve/pii/S0306261924012315>.
- [4] Giarola S, Sachs J, d'Avezac M, Kell A, Hawkes A. MUSE: an open-source agent-based integrated assessment modelling framework. *Energy Strategy Rev* 2022;44:100964. <https://doi.org/10.1016/j.esr.2022.100964>. <https://linkinghub.elsevier.com/retrieve/pii/S22111467X22001584>.
- [5] Schimeczek C, Nienhaus K, Frey U, Sperber E, Sarfarazi S, Nitsch F, Kochems J, Ghazi AAE. AMIRIS: agent-based market model for the investigation of renewable

- and integrated energy systems. *J Open Source Softw* 2023;8(84):5041. <https://doi.org/10.21105/joss.05041>
- [6] GENETX, Hoogsteen G. Utwente-Energy/Alpg. University of Twente - Energy Research Group; Jan 2026. <https://github.com/utwente-energy/alpg>.
- [7] Hofmann F, Hampf J, Neumann F, Brown T, Hörsch J. Atlite: a lightweight Python package for calculating renewable power potentials and time series. *J Open Source Softw* 2021;6(62):3294. <https://doi.org/10.21105/joss.03294>
- [8] Krien U, Schönfeld P. Oemof/Demandlib (Feb, 2026). <https://github.com/oemof/demandlib>, oemof community.
- [9] Constantin L, Barsanti M. Demod (Jan, 2026). <https://github.com/epfl-herus/demod>, epfl-herus.
- [10] Pfeiffer J, Kunick M. Development of an integrated software workflow for district heating network planning: a structured methodological approach. *ACC JOURNAL* 2025;31(1):48–63. <https://doi.org/10.2478/acc-2025-0004>
- [11] Gaete-Morales C, Kramer H, Schill W-P, Zerrahn A. An open tool for creating battery-electric vehicle time series from empirical data, emobpy. *Sci Data* 2021;8(1):152. <https://doi.org/10.1038/s41597-021-00932-9>. <https://www.nature.com/articles/s41597-021-00932-9>.
- [12] FZJ-IEK3-VSA. ETHOS PeNALPS (Sep, 2025). https://github.com/FZJ-IEK3-VSA/ETHOS_PeNALPS, Forschungszentrum Jülich - Jülich Systems Analysis (ICE-2).
- [13] Prataviera E, Romano P, Carnieletto L, Pirotti F, Vivian J, Zarrella A. EURECA: an open-source urban building energy modelling tool for the efficient evaluation of cities energy demand. *Renew Energy* 2021;173:544–60. <https://doi.org/10.1016/j.renene.2021.03.144>. <https://www.sciencedirect.com/science/article/pii/S0960148121005085>.
- [14] Lahariya M, Benoit DF, Devellder C. Synthetic data generator for electric vehicle charging sessions: modeling and evaluation using real-world data. *Energies* Aug 2020;13(16). <https://doi.org/10.3390/en13164211>. <https://www.mdpi.com/1996-1073/13/16/4211>.
- [15] Robinson B, Thibaut W, Nicolas L. HeatPro Python package: generate heat demand load profile for district heating. 2024. <https://github.com/CEA-Liten/HeatPro>, CEA-Liten.
- [16] Charbonnier F, Morstyn T, McCulloch M. Home electricity data generator (HEDGE): an open-access tool for the generation of electric vehicle, residential demand, and PV generation profiles. *Methodsx* 2024;12:102618. <https://doi.org/10.1016/j.mex.2024.102618>. <https://www.sciencedirect.com/science/article/pii/S2215016124000724>.
- [17] Hoops H, Tjaden T, Rösken K. RE-Lab-Projects/Hplib: v1.9 (Jul, 2022). <https://doi.org/10.5281/ZENODO.5521597>. <https://zenodo.org/record/5521597>, Zenodo.
- [18] RWTH-EBC. OpenDHW; 2025. <https://github.com/RWTH-EBC/OpenDHW>.
- [19] Anderson KS, Hansen CW, Holmgren WF, Jensen AR, Mikofski MA, Driesse A. Pvlb Python: 2023 project update. *J Open Source Softw* 2023;8(92):5994. <https://doi.org/10.21105/joss.05994>
- [20] Pflugradt N, Stenzel P, Kotzur L, Stollen D. LoadProfileGenerator: an agent-based behavior simulation for generating residential load profiles. *J Open Source Softw* 2022;7(71):3574. <https://doi.org/10.21105/joss.03574>
- [21] Lombardi F, Balderrama S, Quoilin S, Colombo E. Generating high-resolution Multi-Energy load profiles for remote areas with an open-source stochastic model. *Energy* 2019;177:433–44. <https://doi.org/10.1016/j.energy.2019.04.097>. <https://www.sciencedirect.com/science/article/pii/S0360544219307303>.
- [22] Berkes A, Keshav S. SPAGHETTI: a synthetic data generator for post-covid electric vehicle usage. *Energy Inform* 2024;7(1):15. <https://doi.org/10.1186/s42162-024-00314-6>
- [23] Remmen P, Lauster M, Mans M, Fuchs M, Osterhage T, Müller D. TEASER: an open tool for urban energy modelling of building stocks. *J Build Perform Simul* 2018;11(1):84–98. <https://doi.org/10.1080/19401493.2017.1283539>
- [24] FZJ-IEK3-VSA. Tsib (Sep, 2025). <https://github.com/FZJ-IEK3-VSA/tsib>, Forschungszentrum Jülich - Jülich Systems Analysis (ICE-2).
- [25] Wulff N, Miorelli F, Gils HC, Jochem P. Vehicle energy consumption in Python (VencPy): presenting and demonstrating an open-source tool to calculate electric vehicle charging flexibility. *Energies* Jul 2021;14(14). <https://doi.org/10.3390/en14144349>. <https://www.mdpi.com/1996-1073/14/14/4349>.
- [26] Haas S, Krien U, Schachler B, Bot S, Zeli V, Maurer F, Shivam K, Witte F, Rasti SJ, Bosch S. Wind-Python/Windpowerlib: update release, Zenodo (Feb, 2024). <https://doi.org/10.5281/zenodo.10685057>
- [27] The joblib developers. Joblib (Dec, 2025). <https://doi.org/10.5281/zenodo.14915601>. <https://github.com/joblib/joblib>.
- [28] DIN - German Institute for Standardisation. DIN EN ISO 13790 - Energy performance of buildings calculation of energy use for space heating and cooling; 2008. <https://www.dinmedia.de/de/norm/din-en-iso-13790/103366716>, Norm, Berlin.
- [29] VDI - Association of German Engineers. VDI 6007 - calculation of transient thermal response of rooms and buildings. Tech. rep. Verein Deutscher Ingenieure, Düsseldorf; 2015. <https://www.vdi.de/en/home/vdi-standards/details/vdi-6007-blatt-1-calculation-of-transient-thermal-response-of-rooms-and-buildings-modelling-of-rooms>.
- [30] Jordan U, Vajen K, Kassel U. Dhwcac: program to generate domestic hot water profiles with statistical means for user defined conditions. In: *Proc. ISES solar world congress*. Orlando, FL, USA; 2005.
- [31] Ruhnau O, Hirth L, Praktiknjo A. Time series of heat demand and heat pump efficiency for energy system modeling. *Sci Data* 2019;6(1):189. <https://doi.org/10.1038/s41597-019-0199-y>. <https://www.nature.com/articles/s41597-019-0199-y>.
- [32] Becker V, Kleiminger W. Exploring zero-training algorithms for occupancy detection based on smart meter measurements. *Computer Science - Research and Development* 2018;33(1–2):25–36. <https://doi.org/10.1007/s00450-017-0344-9>
- [33] Zippenfenig P. Open-Meteo.Com weather API, Zenodo (Jan, 2024). <https://doi.org/10.5281/zenodo.7973572>
- [34] Molar-Cruz A, Odersky L, Doepfert M, Meier V. UrbanEnergyPro; 2018. <https://github.com/tum-ens/UrbanHeatPro>.
- [35] Technical University of Munich. NEED. <https://www.epe.ed.tum.de/en/ens/research/projects/current-projects/need/>.
- [36] Buchenberg P, Doepfert M, Reveron Baecker B, Genena HMA, Stengel M, Huang MW, Kalkan K, Kuper L, Ayasse C, Ebrahimi Salari H. infDB: an open-source data ecosystem for urban energy infrastructure modelling and planning (June, 2026). <https://doi.org/10.5281/zenodo.20609622>. <https://zenodo.org/records/20609622>.